

Emerging trends & AI in programming

Programming used to mean typing every line yourself. Now you can describe what you want in plain English and a model writes the code. That changes how we build software — but not whether we still have to understand it. This module shows you the trends, the tools, and the one skill examiners care about most: judging whe

What this key knowledge covers

This summary distils everything you need for **KK01 — Emerging trends & AI in programming** in Programming. Work through the explanations, then use the worked good-vs-weak answers to see exactly how marks are won and lost. Tick off the revision checklist at the end before a SAC or exam.

Study summary

What this dot point is asking

“emerging trends in programming languages and AI-based code generators”

Unit 1, Outcome 2 (Programming). The supporting teaching activity is explicit: you are given code created by an **AI code-generation tool** and asked to **evaluate its correctness** and **compare it to a manually written solution**. So this is not just “name some tools” — it’s a thinking-and-judging skill.

Break the dot point into three examinable threads. Everything below hangs off these:

Where programming languages are heading — higher abstraction , readability-first design, and **low-code / no-code** platforms.

Tools like Copilot, ChatGPT and Gemini that turn a plain-language prompt into code — what they do well and where they quietly fail.

The examinable skill: read AI-written code and decide if it is **correct, readable and maintainable** — or not.

Maya is in Year 11 and is building *QuizQuest* — a small quiz app for her Applied Computing class. She’s on a deadline, so she uses an AI code generator to scaffold it fast. It feels like magic... until a classmate scores exactly 50 and the app marks them as failed. We’ll use Maya’s build to make every idea on this page concrete.

Why programming keeps changing

Every few years, programming gets a new layer that lets you say **more with less**. Punch cards became assembly; assembly became high-level languages; libraries hid the hard parts; and now AI lets you describe intent in English. This single direction — *rising abstraction* — is the spine of this whole dot point.

Think of it like cooking. You can grind your own flour and butcher your own meat (machine code), buy ingredients and follow a recipe (a high-level language), use a meal kit with everything pre-portioned (a framework), or just tell a chef “something warm and Italian” (AI). Each step up trades fine control for speed and convenience. None of them removes the need to know whether the meal is any good.

An **emerging trend** isn’t just “a new app”. In this study it means a **shift in how software gets built** — a new level of abstraction (low-code), a new kind of tool (AI generators), or a new design priority (readability and maintainability over raw cleverness).

Lab 1 — The Abstraction Tower

Here is the *same task* — “add two numbers and show the result” — expressed at six levels of programming history. Click any rung to see how the code changes. Watch the technical detail melt away as you climb. That melting-away **is the trend**.

Words and symbols people understand. Portable across machines. This is where you write Outcome 2 code.

Notice: the bottom rung is unreadable to humans but tells the machine exactly what to do; the top rung is effortless to read but leaves the machine (and the AI) to fill in a lot of assumptions.

More abstraction means more is hidden — which is exactly why you still have to check what the tool produced.

Trends in programming languages

You don’t need to memorise version numbers. You need to recognise the **directions** languages are moving in, and be able to give an example of each.

Languages like **Python** are popular because they read almost like English. Clear code is faster to write, easier to fix, and easier for a team to maintain.

Most modern languages are **OOP** — code is organised into objects that bundle data with the methods that act on it. This is the language family Outcome 2 expects you to use.

Drag-and-drop builders and visual logic (think App Inventor, Power Apps, Scratch-style blocks) let people build working apps with little or no typed code.

Editors now ship with an AI generator built in. Writing code increasingly means *prompting, reading and correcting* as much as typing.

“A current trend in programming languages is the move toward **greater readability and maintainability** (e.g. Python) and toward **higher abstraction** through low-code platforms and AI-based code generators, which let developers produce working solutions with less hand-written code.”

What an AI-based code generator is

An **AI-based code generator** is a tool that produces source code from a natural-language description or from surrounding code. You’ve almost certainly met one: GitHub **Copilot**,

How it actually works (and why that matters)

This is the part most students get wrong in exams. An AI generator does **not** understand your program or “know” it is correct. It is a *prediction engine*: trained on enormous amounts of existing code, it predicts the most statistically likely next tokens for your prompt. It’s a phenomenally good autocomplete — not a tester, and not a guarantee.

An AI code generator answers “**what does code like this usually look like?**” — never “**is this code right for your exact problem?**” That gap is where bugs live.

Because it’s predicting “usual” code, it’s brilliant at common, well-known tasks and shaky at anything unusual, brand-new, or specific to your data. It can also **hallucinate** — confidently invent a function or library that doesn’t exist.

Lab 2 — The AI Code Inspector

Maya asked an AI generator for a function that decides if a quiz mark is a pass. Here’s what it returned:

```
# the pass mark at Maya's school is 50 — a mark of 50 should PASS
```

It reads cleanly and runs without errors — so it’s tempting to trust it. **Don’t.** Launch the test probes below. Each probe carries a mark; it turns green if the AI’s answer matches the correct answer, and red if it doesn’t.

A mark of exactly **50** is excluded, so a genuine pass is marked as a fail. A classic **boundary error** — the code runs perfectly and is still wrong.

Comparing against the manually written, correct version is exactly what the study design asks you to do. One character — `>=` instead of `>` — fixes it.

“Runs without errors” is **not** “correct”. Only **testing against known inputs** — especially **boundary values** like the pass mark itself — exposes the kind of subtle mistake AI generators love to make.

Benefits of AI code generators

Boilerplate, loops and standard structures appear instantly, so you spend time on the hard, specific parts instead of typing routine code.

Seeing a worked solution — and asking “why?” — can teach syntax and patterns, especially for a beginner stuck on where to start.

Generators can rename variables, add comments and restructure messy code, improving **readability and maintainability** — a benefit the study design names directly.

With low-code and AI together, people who can’t yet write much code can still produce a working solution, widening who gets to build software.

Limitations & risks

As Lab 2 showed, code can run cleanly and still be wrong (boundary errors, off-by-one, wrong operator). You must test, not trust.

It can invent functions or libraries that don't exist, or use ones incorrectly — confidently and convincingly.

Generated code may carry insecure patterns; pasting private or school data into a public tool can breach privacy and IP rules.

Output is trained on others' code and may resemble licensed work, raising ownership and copyright questions (the focus of KK06).

Lean on it too early and you may never build the understanding needed to spot when it's wrong.

It doesn't know *your* requirements, constraints or data — only what code “usually” looks like.

Low-code & no-code approaches

Low-code platforms let you build most of an app visually (drag-and-drop screens, point-and-click logic) and drop into written code only when you need something custom.

No-code aims to remove typed code entirely. Both are part of the same trend as AI generators: *get a working solution with less hand-written code.*

Quick prototypes, simple business apps, forms and dashboards, people without a coding background, tight deadlines.

Complex or unusual logic, fine performance tuning, anything the platform doesn't support — you hit a wall the platform's authors didn't plan for.

Trade-off to remember: low-code buys **speed and accessibility** at the cost of **flexibility and control** — the same bargain you make every step up the Abstraction Tower.

Readability & maintainability gains

The study design singles these two out, so define them precisely:

How easily a **human** can understand what code does — clear names, sensible structure, helpful comments.

How easily code can be **changed or fixed later** without breaking things. Readable code is far easier to maintain.

AI tools can push both up (cleaning and commenting code) *or* down (spitting out clever, dense, uncommented code nobody on the team understands). Same input scenario, two outcomes:

Works, but what is *f* ? What is *x* ? Six months later, nobody knows.

Same logic, named and commented. Anyone can read, test and change it.

Evaluating AI-generated code

This is the heart of the dot point. When you're handed AI code in an exam, work a **quick checklist** rather than eyeballing it for vibes:

- **Correctness** — does it produce the right output for normal inputs *and* boundary inputs? (Build a small test table, as in Lab 2.)
- **Requirements fit** — does it actually do what the brief asked, including constraints and edge cases?
- **Readability** — clear names, structure and comments, or a cryptic one-liner?
- **Maintainability** — could another student safely change it next term?

- **Compare to a manual solution** — how does it differ from how you'd write it by hand? Which is better, and why?

P-E-L writing trainer

Short-answer marks come from **structure**, not word-count. Use **P-E-L — Point, Example, Link**. Make your point, prove it with a concrete example, link it back to the question. Click each tab for a worked sentence on “a limitation of AI code generators”.

Make a clear, examinable claim. No hedging.

Anchor the point in something concrete and specific.

Tie it back to the command word (here, a limitation's consequence).

VCE-style practice questions

Attempt each on paper first. Then reveal the weak vs strong responses and the marking guide, and self-mark — your score feeds the dock in the corner.

- 1 mark: correct definition of an AI code generator (prompt/code → generated code).
- 1 mark: a valid, named trend (readability-first / low-code / higher abstraction / AI-assisted).
- 1 mark per benefit identified (max 2).
- 1 mark for explaining the effect/consequence of one benefit.
- Valid benefits: speed/scaffolding, learning, readability/maintainability, lower barrier.
- 1 mark per limitation described (max 2).
- 1 mark per matching consequence (max 2).
- Limitations: silent bugs, hallucination, security/privacy, IP/licensing, over-reliance, no context.
- 1 mark: identifies it runs but is logically incorrect.
- 1 mark: pinpoints > vs >= boundary error at mark = 50.
- 1 mark: explains effect (real passes marked as fails).
- 1 mark: describes testing with boundary values.
- 1 mark: compares to the correct manual solution (>=).
- 1-2 marks: relevant trends (rising abstraction, low-code, AI generators).
- 1-2 marks: benefits with example (speed, accessibility, scaffolding).
- 1-2 marks: limitations with example (logic bugs, no context, low-code ceiling).
- 1 mark: reasoned conclusion that the role evolves rather than disappears.
- Hold a mark for a one-sided answer with no conclusion.

One-screen cheat sheet

Digital Vector · VCE Applied Computing Units 1 & 2 · U1·O2 Programming · KK01 Single-file & fully offline · Three.js r128 inlined · en-AU · Built for Year 11 & 12.

Common traps & misconceptions

Exam trap

T1 “It ran, so it's correct.” Running without errors only proves the *syntax* is valid. Correctness is about *output*, proven by testing. → Always mention testing, especially boundary values.

Exam trap

T2 Saying AI “understands” code It predicts likely tokens from training data; it has no understanding or guarantee of correctness. → Describe it as a prediction/pattern tool, not a thinker.

Exam trap

T3 Only listing benefits (or only risks) “Discuss/evaluate” questions need **both sides** plus a judgement. → Give benefits and limitations, then a reasoned conclusion.

Exam trap

T4 Confusing readability with maintainability Readability = easy to *understand* now; maintainability = easy to *change* later. Related, not identical. → Define each separately if both are asked.

Exam trap

T5 “Low-code = no skill needed” Low-code still needs logic, design and testing; it removes *typing*, not *thinking*. → Frame it as less hand-written code, not no expertise.

Exam trap

T6 Vague answers with no example “AI is fast and helpful” scores little. Name a tool, a trend, a concrete benefit/limit. → Anchor every point to a specific example.

How to answer exam questions

Read the **command word** first — it sets how much to write. *State/Identify* wants a word or phrase; *Describe* wants features; *Explain* wants reasons and links; *Justify/Evaluate* wants a judgement backed by evidence. Always pull in the **scenario detail** rather than answering in the abstract. The examples below contrast a weak response with a strong one for the same question.

Q1. Define

. Define what is meant by an **AI-based code generator**, and identify one current trend in programming languages.

✗ A weak answer

It's an AI that makes code for you, like ChatGPT. Coding is getting more advanced. Examiner: Too vague. No real definition of how it works, and 'more advanced' isn't a named trend.

✓ A strong answer

An AI-based code generator is a tool that produces source code from a natural-language prompt or surrounding code by predicting the most likely code, e.g. GitHub Copilot. A current trend in programming languages is the move toward greater readability and maintainability, such as the popularity of Python. Examiner: Clear definition (incl. how it works) + a named tool, plus a specific, correctly named trend.

How the marks are awarded

- 1 mark: correct definition of an AI code generator (prompt/code generated code).
- 1 mark: a valid, named trend (readability-first / low-code / higher abstraction / AI-assisted).

Q2. Explain

. Maya uses an AI code generator while building QuizQuest. **Explain two benefits** this could provide.

✗ A weak answer

It's faster and it helps her code. That's good because she saves time. Examiner: Two ideas but they're basically the same point, and 'helps her code' isn't explained.

✓ A strong answer

First, it scaffolds routine code (loops, input handling) instantly, so Maya spends her limited time on the quiz logic specific to her brief. Second, it can improve readability and maintainability by adding clear names and comments, making her solution easier for a marker — or future Maya — to follow and change. Examiner: Two distinct, well-explained benefits, each linked to a consequence for Maya.

How the marks are awarded

- 1 mark per benefit identified (max 2).
- 1 mark for explaining the effect/consequence of one benefit.
- Valid benefits: speed/scaffolding, learning, readability/maintainability, lower barrier.

Q3. Describe

. AI generators are not always trustworthy. **Describe two limitations**, and for each explain a possible consequence for a software solution.

✗ A weak answer

It can be wrong and it can copy other people's code. That's bad for the program. Examiner: Two limitations are hinted at but neither consequence is actually described.

✓ A strong answer

First, generated code can contain silent logic errors — code that runs but is wrong, like using `>` instead of `>=`. The consequence is incorrect behaviour (a pass mark of 50 wrongly fails), which testing must catch. Second, it can reproduce others' licensed code, raising IP/copyright issues; the consequence is that the solution may not be legally safe to distribute. Examiner: Two clearly different limitations, each with a specific, correct consequence.

How the marks are awarded

- 1 mark per limitation described (max 2).
- 1 mark per matching consequence (max 2).
- Limitations: silent bugs, hallucination, security/privacy, IP/licensing, over-reliance, no context.

Q4. Evaluate

. An AI generator returns this function for QuizQuest: `def is_pass(mark): if mark > 50: return True return False` The pass mark is 50. **Evaluate the correctness** of this code: identify the problem, explain its effect, and state how you would verify it against a manual solution.

✗ A weak answer

The code is wrong because it uses `> 50`. It should be fixed. I'd just read it again to check.

Examiner: Identifies the operator but doesn't explain the boundary effect, and 'read it again' isn't a verification method.

✓ A strong answer

The code is syntactically valid but logically incorrect. It uses `'mark > 50'`, so a mark of exactly 50 returns False even though 50 is a pass — a boundary error. The effect is that genuine passes are recorded as fails, corrupting results. To verify, I'd build a test table with inputs above, at, and below the boundary (e.g. 80pass, 50pass, 30fail) and run it; the manual solution uses `'mark >= 50'`, which passes all three tests, confirming the AI version is wrong at the boundary.

Examiner: Identifies the boundary bug, explains the consequence, and verifies with a test table against the corrected manual version — exactly the study-design skill.

How the marks are awarded

- 1 mark: identifies it runs but is logically incorrect.
- 1 mark: pinpoints `>` vs `>=` boundary error at mark = 50.
- 1 mark: explains effect (real passes marked as fails).
- 1 mark: describes testing with boundary values.
- 1 mark: compares to the correct manual solution (`>=`).